

基于熵模型的动态粒子群优化算法

李宏光, 廉莹, 方梦琪

(北京化工大学 信息科学与技术学院, 北京 100029)

摘 要: 受多种群并行寻优机制的启发,提出了一种基于熵模型的动态粒子群优化算法(entropy dynamic multi-PSO, EDM-PSO)用于处理动态优化问题. 将解空间划分为多个子空间,在每个子空间中利用熵模型增加种群多样性,多种群并行搜索,利用多点环境检测机制检测环境变化. 对动态多峰 benchmark 优化问题进行了数值实验,并与其他几种动态优化算法进行了比较,结果表明:EDM-PSO 算法对于处理动态优化问题具有优势.

关键词: 粒子群优化算法; 动态优化; 熵模型

中图分类号: TP29

文献标志码: A

文章编号: 0254-0037(2015)05-0657-05

doi: 10.11936/bjutxb2014100042

Entropy-based Dynamic Particle Swarm Optimization Algorithm

LI Hong-guang, LIAN Ying, FANG Meng-qi

(College of Information Science & Technology, Beijing University of Chemical Technology, Beijing 100029, China)

Abstract: Inspired by the multi-population parallel optimization mechanism, this paper proposes an Entropy-based Dynamic Multi-population Particle Swarm Optimization (EDM-PSO) algorithm which can be utilized to deal with dynamic optimization problems. The solution space was divided into multiple sub-spaces, in which the entropy models were utilized in each sub-space to increase the diversity of populations. Additionally, the multi-population parallel searching mechanism and multi-point detection mechanism were also implemented to seek the optimal solution and to detect ambient environmental changes respectively. Finally, a comparison between EDM-PSO and several other dynamical optimization algorithms in terms of the errors (standard deviation) when addressing a moving peaks function benchmark problem was made, resulting in that the EDM-PSO algorithm can be more beneficial to solving dynamic problems.

Key words: particle swarm optimization; dynamic optimization; entropy model

近年来,智能优化技术正在受到越来越广泛的关注. Kennedy 等^[1]受鸟群觅食行为的启发提出了粒子群优化算法(particle swarm optimization, PSO),由于其容易理解、易于实现,因此已经成为研究和应用的热点.

然而,基本的粒子群优化算法并不具备对动态环境的检测和响应能力,为此,一些学者对其进行了

改进以适应动态环境的优化问题处理. Ratnaweera 等^[2-3]将线性调整参数引入动态 PSO 算法中,从而增强其搜索的效用;另外,一些文献也研究了动态 PSO 算法的应用. Rakitianskaia 等^[4]将动态粒子群算法应用于动态环境变化下的神经网络训练过程中;Kardam 等^[5]将动态粒子群算法用于动态环境下的发动机生成系统调度问题,可以有效降低燃烧

收稿日期: 2014-10-17

基金项目: 中央高校基本科研项目(YS1404)

作者简介: 李宏光(1963—),男,教授,主要从事复杂大系统协调优化、工业过程智能控制方面的研究, E-mail: lihg@mail.

buct.edu.cn

成本.

PSO 动态环境寻优的关键问题是保持种群的多样性,许多学者提出了相应的解决方案. 其中,比较好的方法是通过限制解方案之间的交流来降低算法收敛到全局最优的速度,从而保持环境的多样性. Janson 等^[6]提出了 HPSO 算法,利用分层和伪树结构来保证可行解的多样性;Hashemi 等^[7]提出了细胞 PSO 算法,通过对局部信息进行交换以及利用细胞间自动分布的特征来保证多样性,继而,他们又对环境变化后一些粒子进行量化处理,从而提高了算法整体效用^[8]. 虽然这些算法都在一定程度上增加了算法的多样性,但是并没有建立量化的检测机制来评价和指导种群的分布情况,使得寻优结果受到影响.

本文提出了一种基于熵模型的动态粒子群优化算法 EDM-PSO,利用解空间等分机制来实现多种群并行搜索,提高了算法的效率. 构造了熵模型以解决多样性的保持问题,熵模型很好地检测出种群中粒子的分布情况,以采取相应的措施,并在环境变化之后利用多点检测机制来进行检测.

1 EDM-PSO 算法

1.1 空间等分多种群初始机制

PSO 算法将鸟群中的每一只鸟表示成一个没有质量的“粒子”,而食物便是“最优解”. 假设粒子的初始种群的数量为 N ,则第 i ($i=1,2,\dots,N$) 个粒子的位置信息可以用 X_i 来表示,那么它所遍历过的“最优”的位置就记为 p_{best} ,用 V_i 来表示此粒子的速度,将群体中“最优”的粒子的位置标号表示为 G_{best} . 由此可见粒子 i 可以根据

$$\begin{aligned} V_i &= w \times V_i + c_1 \times \text{Rand}() \times (p_{\text{best}} - X_i) + \\ &\quad c_2 \times \text{Rand}() \times (G_{\text{best}} - X_i) \\ X_i &= X_i + V_i \end{aligned} \quad (1)$$

来更新其自身的位置和速度.

对于复杂的多峰函数来说,在解空间中分布着多个峰值,利用传统的 PSO 算法来进行优化时,很容易陷入局部极值,而为了找到全局最优值,需要初始种群足够庞大,这样就增加了算法的复杂度. 为了解决这个问题,将解空间等分为 n 段子空间,在每个子空间中分别初始化等数目的种群,并定义相同的最大迭代次数,具体算法步骤如下:

步骤 1 将解空间 D 分为等分的子空间 $d_1, \dots, d_n$, 定义环境最大变化次数 T_{Envir} .

步骤 2 在每个子空间中随机初始化 N 个粒

子,其速度与位置分别为 V_n 和 X_n ,计算粒子的适应度函数值 f_n ,并定义最大迭代次数 T_m ,得出当前子空间中种群最优粒子适应度函数值 G_{bestn} .

步骤 3 在每个子空间中利用式(1)更新粒子的速度和位置. 计算得出每个粒子的个体极值 p_{bestn} 和子空间的全局极值 G_{bestn} .

1.2 基于熵模型的多样性增强机制

信息熵模型是由 Shannon 于 1948 年提出的,主要用于测量各种信息概率分布的混乱程度^[9]. 由于在基本粒子群算法中,各粒子的运动速度与方向均会受到全局最优位置的影响,从而会不可避免地出现过度聚集或者过度分散现象. 其中,前者将导致种群的多样性下降,陷入局部最优;而后者则会降低粒子的搜索效率. 因此,提出引入熵模型可以有效地对粒子初始时的整体分布情况进行检测,并对粒子进行动态调整,从而更为快速、准确地搜索到最优解. 这里,构造的熵模型采用

$$S(t) = - \sum_{i=1}^N p(x_{ii}) \log_2 p(x_{ii}) \quad (2)$$

式中: $S(t)$ 为第 t 代的熵值; N 为整个种群的粒子总数; $p(x_{ii})$ 为第 t 代中第 i 个粒子的权重值,且有 $\sum_{i=1}^N p(x_{ii}) = 1, 0 < p(x_{ii}) < 1$. 利用粒子的适应度值比例对 $p(x_{ii})$ 进行构造,即

$$p(x_{ii}) = \frac{g_{ii}}{\sum_{i=1}^N g_{ii}} \quad (3)$$

式中 g_{ii} 为第 t 代中第 i 个粒子的适应度值. 因此, $S(t)$ 越大说明各粒子的适应度值越相似,越接近最优解,即越聚集;反之说明各粒子的适应度值之间的差别越大,即越分散. 相应的粒子调整步骤如下:

步骤 1 定义熵值的区间 $[S(t)_{\text{max}}, S(t)_{\text{min}}]$, 且

$$S(t)_{\text{max}} = \begin{cases} 1 - \frac{\mu_1(t-t^*)^2}{T^2} & t \leq t^* \\ 1 & t > t^* \end{cases} \quad (4)$$

$$S(t)_{\text{min}} = \mu_2 t \quad (5)$$

式中 μ_1, μ_2 与 t^* 均为常数.

步骤 2 当熵值过大或过小时,即粒子分布没有处于理想状态时,采用

$$\begin{aligned} v_{id}(k+1) &= C_s w \times v_{id}(k) + C_1 R_1 [p_{id} - x_{id}(k)] + \\ &\quad C_2 R_2 [p_{gd} - x_{id}(k)] \end{aligned} \quad (6)$$

对粒子速度进行调整. 式中 C_s 可表示为

$$C_s =$$

$$\begin{cases} 1 + \lambda_{\max} [S(t) - S(t)_{\max}] & S(t) > S(t)_{\max} \\ 1 & S(t)_{\min} \leq S(t) \leq S(t)_{\max} \\ 1 - \lambda_{\min} [S(t)_{\min} - S(t)] & S(t) < S(t)_{\min} \end{cases} \quad (7)$$

式中 $\lambda_{\max}$ 和 $\lambda_{\min}$ 均为调整参数。

1.3 算法描述

动态优化问题中环境是变化的,算法是否可以及时有效地检测到环境的变化是评价优化算法的重要指标。为了有效地检测环境的变化,文中提出的算法实行了多点检测机制。在算法初始时将解空间分为 n 段,在每一段中都存在一个种群,整个解空间 D 形成了多种群并行搜索的形式,通过比较每个种群中的最优值 $G_{\text{best}1}, \dots, G_{\text{best}n}$ 得到全局最优值 G_{best} ,在检测环境是否发生变化时,只需要不断地将子空间中最优点 $G_{\text{best}1}, \dots, G_{\text{best}n}$ 的值带入函数中,任何一个 G_{best} 发生变化,说明环境发生了变化。

EDM-PSO 算法描述如下:

步骤1 将解空间 D 分为等分的子空间 $d_1 \dots d_n$, 定义环境最大变化次数 T_{Envir} 。

步骤2 在每个子空间中随机初始化 N 个粒子的速度 V_n 、位置 X_n , 计算粒子的适应度函数值 f_n , 定义最大迭代次数 T , 得出当前种群最优粒子适应度函数值 $G_{\text{best}n}$ 。

步骤3 在每个子空间中利用式(1)更新粒子的速度和位置。计算得出每个粒子的个体极值 $p_{\text{best}n}$ 和整体的全局极值 $G_{\text{best}n}$ 。

步骤4 利用分布熵公式(2)进行计算,得出当前种群的熵值 S_n , 判断该时刻的熵值是否满足多样性判别条件公式(3), 如果满足进行下一次迭代, 否则进行步骤5。

步骤5 根据多样性调整公式(4)对种群中个体速度进行调整,直到熵值满足多样性判别条件,更新粒子的速度。若达到最大迭代次数 T , 输出每个子空间的全局极值 $G_{\text{best}n}$, 否则转向步骤3, 继续迭代。

步骤6 比较所有子空间输出得到的 $G_{\text{best}n}$, 得到函数的全局最优值 G_{best} , 若达到环境最大变化次数 T_{Envir} , 输出全局极值, 否则进行步骤7。

步骤7 检查所有子空间中的最优值 $G_{\text{best}n}$ 是否发生变化, 如果没有变化, 则继续检查; 如果至少有一个发生变化, 进行步骤5。

1.4 复杂度分析

算法复杂度分为时间复杂度和空间复杂度。其中, 时间复杂度是指执行算法所需要的计算工作量;

而空间复杂度是指执行这个算法所需要的内存空间。

1.4.1 时间复杂度分析

基于 EDM-PSO 算法的基础语句, 对其时间复杂度进行分析如下:

T_1 初始解空间

T_2 定义最大环境变化次数 T_{Envir} ;

T_3 for ($i = 1$; $i < T_{\text{Envir}} + 1$; $i++$)

// 如果没有达到最大环境变化次数

{

T_4 定义等分解空间个数 d ;

T_5 for ($j = 1$; $j < d + 1$; $j++$)

// 如果没有达到最大空间等分个数

{

T_6 初始 N 个粒子, 定义最大迭代次数 T_m ;

T_7 for ($k = 1$; $k < T + 1$; $k++$)

// 如果没有达到最大迭代次数

{

T_8 计算适应度值;

T_9 更新粒子速度和位置;

T_{10} 计算熵值;

T_{11} 执行多样性调整公式直到满足设

定条件;

}

T_{12} 输出子空间最优值;

}

T_{13} 输出全局最优值;

}

T_{14} 输出所有动态最优值;

可见, 该算法包括 3 个循环, 其时间复杂度为

$$\begin{aligned} & T_1(n) + T_2(n) + T_3(n) \cdot (T_4(n) + \\ & T_5(n) \cdot (T_6(n) + T_7(n) \cdot \\ & (T_8(n) + T_9(n) + T_{10}(n) + T_{11}(n)) + \\ & T_{12}(n)) + T_{13}(n)) + T_{14}(n) = \\ & O(T_{\text{Envir}} \cdot (d \cdot (N \cdot T))) \end{aligned}$$

1.4.2 空间复杂度

与时间复杂度类似, 空间复杂度是指算法在计算机内执行时所需空间的度量, 记作 $O(f(n))$ 。一个算法在计算机存储器上所占用的存储空间由输入输出数据所占用的存储空间决定。因此, EDM-PSO 算法的空间复杂度可以表示为 $O(d \cdot N)$ 。

2 实例分析

2.1 动态多峰 benchmark 问题

考虑 1999 年 Jurgen^[10] 针对动态优化测试给出

的动态多峰 benchmark 问题,它的坐标空间为 5 维,峰的数目为 5,动态改变 5 峰的高、宽和坐标.记 t 为变化次数,5 峰的高、宽和坐标分别记为 $h_i(t)$ 、 $w_i(t)$ 和 $p_i(t)$, $p_i(t) = (p_i^1(t), \cdots, p_i^5(t))$, $i = 1, 2 \cdots, 5$. 5 峰的初值见表 1.

表 1 动态多峰 benchmark 问题 5 峰的初值
Table 1 Initial values of five peaks within the moving peaks function benchmark problem

序号	$p_i^1(t)$	$p_i^2(t)$	$p_i^3(t)$	$p_i^4(t)$	$p_i^5(t)$	$w_i(t)$	$h_i(t)$
峰 1	8	64	67	55	4	0.1	50
峰 2	50	13	76	15	7	0.1	50
峰 3	9	19	27	67	24	0.1	50
峰 4	66	87	65	19	43	0.1	50
峰 5	76	32	43	54	65	0.1	50

记 σ 是服从标准正态分布的随机数, ν 是模为 0.06 的随机向量,5 峰动态改变为

$$\begin{cases} h_i(t) = h_i(t-1) + 7\sigma, h_i(t) \in [30, 70], \\ w_i(t) = w_i(t-1) + 0.01\sigma, w_i(t) \in [0.8, 7], \\ p_i(t) = p_i(t-1) + \nu, p_i^j(t) \in [0, 100] \end{cases}$$

(8)

式中 $\sigma \in N(0, 1)$.

由此设置动态多峰 benchmark 问题模型的目标函数表达式为

$$F(x, t) = \max_{1 \leq i \leq 5} \frac{h_i(t)}{1 + w_i(t) \sum_{j=1}^5 (x^j - p_i^j(t))^2},$$
$$x \in G = \{ (x^1, x^2, \cdots, x^5) | x^j \in [30, 70], j = 1, 2, \cdots, 5 \}$$

(9)

求解的目的是在可行域 G 中寻找使目标函数 $F(x, t)$ 取得最大值的最优解.

这是一个全局最优值预先确知的问题模型. 因为第 t 次动态变化后,当解 $x = p_i(t)$, $i = 1, 2, \cdots, 5$ 时,得到目标函数 $F(x, t)$ 的极大值 $h_i(t)$,求得全局的最大值 $\max_{1 \leq i \leq 5} h_i(t)$. 求解的难点是动态变化后,当群体的所有个体全部转化成较差解时,算法必须在下一次动态变化到来之前的有限间隔内寻找到最优解或近似最优解.

2.2 EDM-PSO 算法参数设置

解空间分为 4 个子空间,在每个子空间中随机初始化 20 个粒子,最大迭代次数为 100,环境变化的最大次数为 9 次;在式(1)中,权重因子 w 取 0.4,学习因子 c_1 和 c_2 均取 2. 式(4)(5)中取 $\mu_1 = 200, \mu_2 =$

0.4, $t^* = 0.7$. 式(7)中 $\lambda_{\max}$ 取 1.3, $\lambda_{\min}$ 取 0.7.

2.3 实验结果及分析

EDM-PSO 算法运行的条件是环境变化 9 次,每次都得到一个当前环境下的最优值,表 2 中列出了每次环境变化之后真实值和算法所能搜索到的最优值对比以及相对误差和绝对误差. 图 1 为 9 次环境变化条件下寻优的曲线图. 从数据信息中可以看出,真实值和最优值得到的绝对误差值大多数情况下在合理的范围内,少数几次偏差稍大,可能是由算法的随机性造成的,综合评价算法在每次环境变化之后都可以逼近真实值.

表 2 环境变化时真实值和算法搜索到的最优值对比
Table 2 Comparison between the optimal values gained through the algorithm and the true values with the changes of environment

环境变化	真实值	最优值	绝对误差	相对误差
0	50.000	49.918	0.082 3	0.001 6
1	59.623	58.983	0.640 9	0.010 7
2	66.416	66.271	0.145 2	0.002 2
3	68.803	67.961	0.841 9	0.012 2
4	69.692	67.130	2.561 6	0.036 8
5	70.000	69.462	0.538 2	0.007 7
6	70.000	69.627	0.372 6	0.005 3
7	70.000	68.299	1.701 3	0.024 3
8	69.579	66.435	3.143 7	0.045 2
9	69.567	68.069	1.497 6	0.021 5

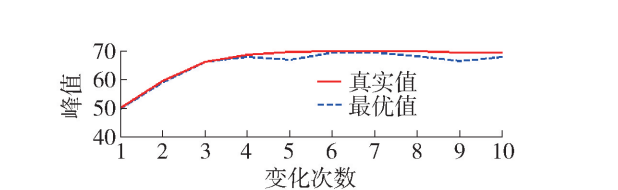


图 1 9 次环境变化条件下寻优曲线
Fig. 1 Optimizaiton curve under 9 times of changes of environment

为了验证 EDM-PSO 算法动态寻优的能力,表 3 列出了与另外 5 个动态寻优的算法的比较^[11-15],峰值数量均为 5 峰,其他参数的设定和文中之前设定的相同,表中列出了各种方法搜索到的最优值和真实值之间的误差以及每种方法中算法运行迭代过程的平均标准差. 可见,本文提出的算法可以更加准确地找出函数的最优值.

可以看出,所提出的 EDM-PSO 算法在环境变化之后能准确地寻找到最优值,并且,标准差的比较

表3 在峰值数量为5的情况下,6种算法的
寻优误差(标准差)的比较

Table 3 Comparison of the optimization errors (standard deviations) of six algorithms when the number of peaks is 5

算法	峰值数量	误差	标准差
mQSO[13]	5	11.91	0.76
AmSO[14]	5	5.77	0.56
mPSO[15]	5	6.69	0.26
HmPSO[16]	5	7.40	0.31
APSO[17]	5	4.95	0.11
EDM-PSO	5	1.50	0.32

显示了算法的寻优平稳性.

3 结论

为了解决动态多峰函数优化问题,在所提出的 EDM-PSO 算法中建立了3种机制. 等分空间机制执行多种群并行搜索的方法,解决了空间复杂度的问题;利用熵模型来检测粒子的分散程度,保持了种群寻优的多样性;动态环境检测机制利用多点检测原理,增加了检测到环境变化的成功率. 对动态多峰 benchmark 问题进行了处理,并与其他5种动态寻优算法进行比较,结果显示了所提出的算法具有良好的收敛性和稳定性.

参考文献:

[1] KENNEDY J, EBERHART R. Particle swarm optimization [C] // Proc IEEE Int Conf Neural Networks. [S. l.]: IEEE, 1995: 1942-1948.

[2] RATNAWEERA A, HALGAMUGE S, WATSON H C. Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients [J]. Evolutionary Computation, IEEE Transactions on, 2004, 8(3): 240-255.

[3] CHANG B C H, RATNAWEERA A, HALGAMUGE S K, et al. Particle swarm optimization for protein motif discovery [J]. Genetic Programming and Evolvable Machines, 2004, 5(2): 203-214.

[4] RAKITIANSKAIA A, ENGELBRECHT A P. Training neural networks with PSO in dynamic environments[C] // Evolutionary Computation, 2009, CEC'09. IEEE Congress on. Trondheim: IEEE, 2009: 667-673.

[5] KARDAM N, ANSARI M A, FARHEEN F.

Communication and load balancing using SCADA model based integrated substation [C] // Energy Efficient Technologies for Sustainability (ICEETS), 2013 International Conference on. Nagercoil: IEEE, 2013: 1256-1261.

[6] JANSON S, MIDDENDORF M. A hierarchical particle swarm optimizer for dynamic optimization problems [M] // Applications of Evolutionary Computing. [S. l.]: Springer Berlin Heidelberg, 2004: 513-524.

[7] HASHEMI A B, MEYBODI M R. Cellular PSO: a PSO for dynamic environments [M] // Advances in computation and intelligence. [S. l.]: Springer Berlin Heidelberg, 2009: 422-433.

[8] HASHEMI A B, MEYBODI M R. A multi-role cellular PSO for dynamic environments[C] //14th International CSI Computer Conference, CSICC 2009. Tehran: IEEE, 2009: 412-417.

[9] SHANNON C E. A mathematical theory of communication [J]. ACM SIGMOBILE Mobile Computing and Communications Review, 2001, 5: 3-55.

[10] JURGEN B. Memory enhanced evolutionary algorithms for changing optimization problems[C] // Proceedings of the 1999 Congress of Evolutionary Computation. Washington, D. C. : IEEE, 1999: 875-1882.

[11] BLACKWELL T, BRANKE J. Multi-swarms, exclusion, and anti-convergence in dynamic environments[J]. IEEE Transactions on Evolutionary Computation, 2006, 10: 459-472.

[12] BLACKWELL T, BRANKE J, LI X. Particle swarms for dynamic optimization problems[M] // Swarm Intelligence. [S. l.]: Springer Berlin Heideberg, 2008: 193-217.

[13] KAMOSI M, HASHEMI A B, MEYBODI M R. A new particle swarm optimization algorithm for dynamic environments[M] // Swarm, Evolutionary, and Memetic Computing. Chennai: Springer Berlin Heidelberg, 2010: 129-138.

[14] KAMOSI M, HASHEMI A B, MEYBODI M R. A hibernating multi-swarm optimization algorithm for dynamic environments [C] // Proceedings of World Congress on Nature and Biologically Inspired Computing (NaBIC2010). Fukuoca: IEEE, 2010: 370-376.

[15] REZAZADEH I, MEYBODI M R, NAEBI A. Adaptive particle swarm optimization algorithm for dynamic environments [M] // Advances in Swarm Intelligence. Chongqing: Springer Berlin Heidelberg, 2011: 120-129.

(责任编辑 张 蕾)